

Coding & Solution

Date	03 October 2026
Team ID	4
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Coding & Solution

FitBuddy implements all four scenarios of the project brief and fixes the brief's known defects: manually typed user IDs, ignored age/weight, overwritten revisions, raw markdown output and error text saved as plans.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/sharukeshalagiri/FitBuddy
Programming Language(s)	Python 3.11+, HTML5, CSS3, Jinja2 templating
Framework(s) Used	FastAPI, Uvicorn, SQLAlchemy 2.x, Pydantic v2, google-genai SDK
Key Features Implemented	- Personalized 7-day plan (age, weight, goal, intensity, experience) as structured JSON → day cards - Nutrition / recovery tip (light model, built-in fallback tip) - Feedback revision of the latest version with full version history - Password-protected admin dashboard with original vs latest plan and delete - JSON API with Swagger docs; health endpoint - Retry + automatic fallback model; demo mode with no API key
Pending / Incomplete Features	None from the planned scope. Future scope: meal plans, progress tracking, cloud deployment (see Scalability & Future Plan).
Setup / Run Instructions	<pre>python -m venv venv venv\Scripts\activate (Mac/Linux: source venv/bin/activate) pip install -r requirements.txt copy .env.example .env (then add GEMINI_API_KEY; optional) uvicorn app.main:app --reload Open http://127.0.0.1:8000 API docs: http://127.0.0.1:8000/docs</pre>

Core code excerpt (app/gemini_generator.py)

```
def build_prompt(goal: str, intensity: str, experience: str, age: Optional[int],
                weight: Optional[float]) -> str:
    lines = [
        "Create a 7-day workout plan for this person:",
        f"- Fitness goal: {goal}",
        f"- Workout intensity: {intensity}",
        f"- Experience level: {experience}",
        f"- Age: {age if age is not None else 'not provided'}",
        f"- Body weight: {f'{weight:g} kg' if weight is not None else 'not provided'}",
    ]
    return "\n".join(lines)

def generate_workout_gemini(goal: str, intensity: str, experience: str = "beginner",
                           age: Optional[int] = None,
                           weight: Optional[float] = None) -> tuple[dict, str, str]:
    """Return (plan_dict, source, model). Raises GeminiError on AI failure (never error
    text)."""
    if config.DEMO_MODE:
        return demo_data.demo_workout_plan(goal, intensity, experience, age, weight),
        "demo", "demo"
    plan, model = generate_structured(config.GEMINI_WORKOUT_MODEL, SYSTEM_PROMPT,
                                     build_prompt(goal, intensity, experience, age,
                                     weight),
                                     WorkoutPlan, config.GEMINI_WORKOUT_FALLBACK_MODEL)
    return plan.model_dump(), "gemini", model
```

Code Quality Checklist

S.No	Criteria	Status
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- 35 automated tests (pytest) pass in demo mode, covering every HTML route, JSON endpoint, validation, admin auth, versioning and the fallback model.
- Real-Gemini run verified: schema accepted, every plan passed validation; revisions applied feedback (e.g. "no equipment" switched to bodyweight moves).
- The project brief's Gemini 1.5 models and google-generativeai SDK are retired/deprecated; FitBuddy uses current models and the google-genai SDK.
- Development was assisted by an AI coding assistant (Claude Code); the team specified, reviewed and tested the implementation.